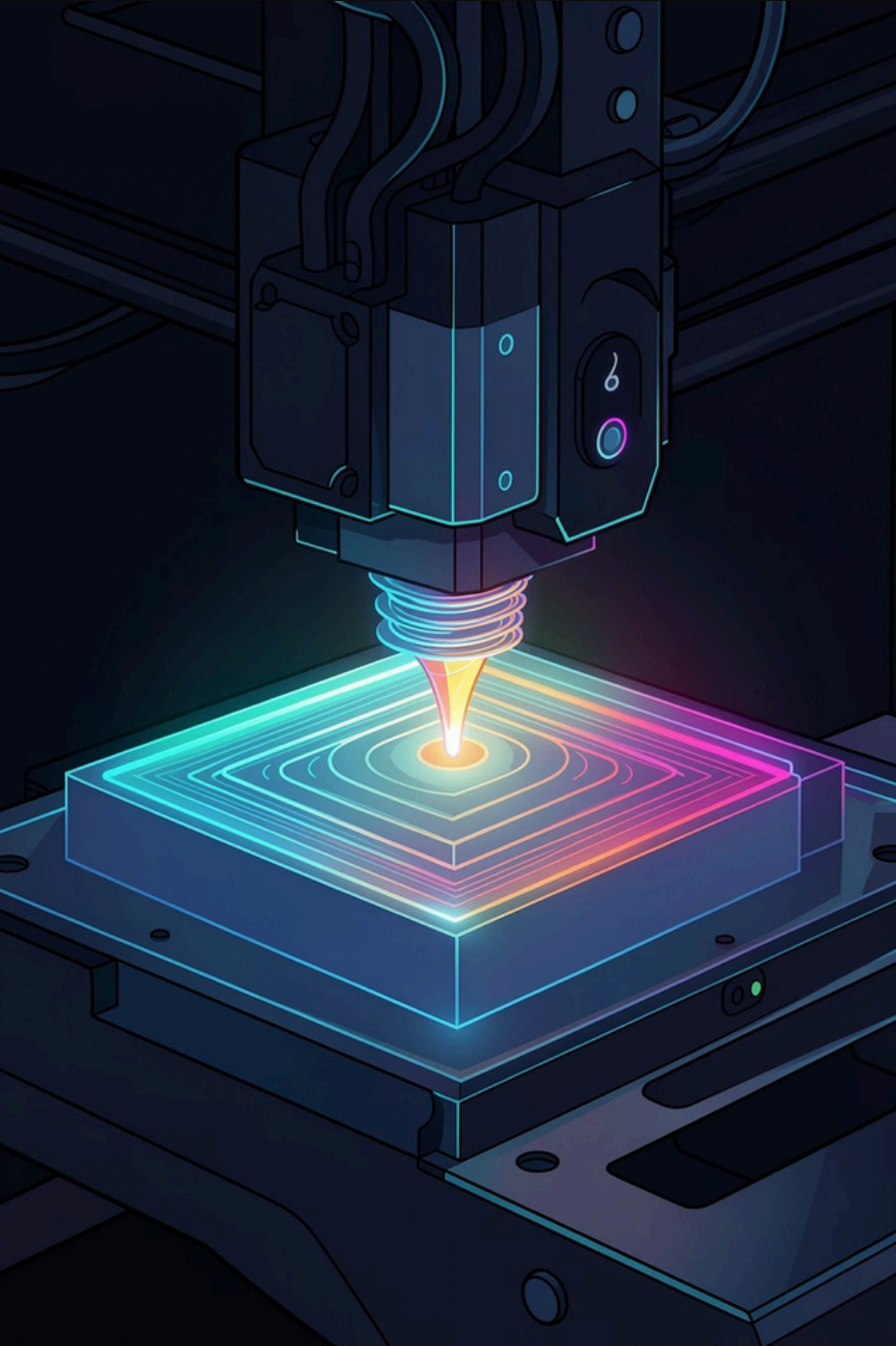




AI-Driven Defect Detection in 3D Printing

Closed-Loop Real-Time Monitoring System

PRESENTED BY SOUMYA RANJAN SENAPATI

GUIDED BY PROF. SUSHANT KUMAR SAHOO

Model Training

Training a YOLOv26l Model

Training Configuration

- Trained for 96 total epochs
- Using Kaggle GPU (NVIDIA P100/T4)
- Ultralytics + Roboflow framework
- Dataset: 10,219 total images
- 8,694 training images
- 1,527 validation images

1

Final Model Metrics

mAP50-95: 0.3894

mAP50: 0.6085

Precision: 0.7363

Recall: 0.5807

2

Key Insights

- High precision (73.63%) means minimal false alarms
- Strong recall (58.07%) ensures most defects are caught
- mAP50 of 60.85% demonstrates reliable detection across confidence thresholds
- Model ready for real-time deployment

Objectives — Closing the Loop

1

Model Upgrade

Retrain using the larger **YOLOv26l** architecture for deeper feature extraction and improved accuracy

2

Live Deployment

Stream real-time video from a **mobile phone IP camera** over WiFi to monitor the Ender 3 build plate

3

Defect Induction

Deliberately misconfigure the Ender 3 to **reproducibly generate Stringing** defects for controlled validation

4

Alert System

Python script detects defects and **auto-sends an email** with a snapshot photo to the user upon detection

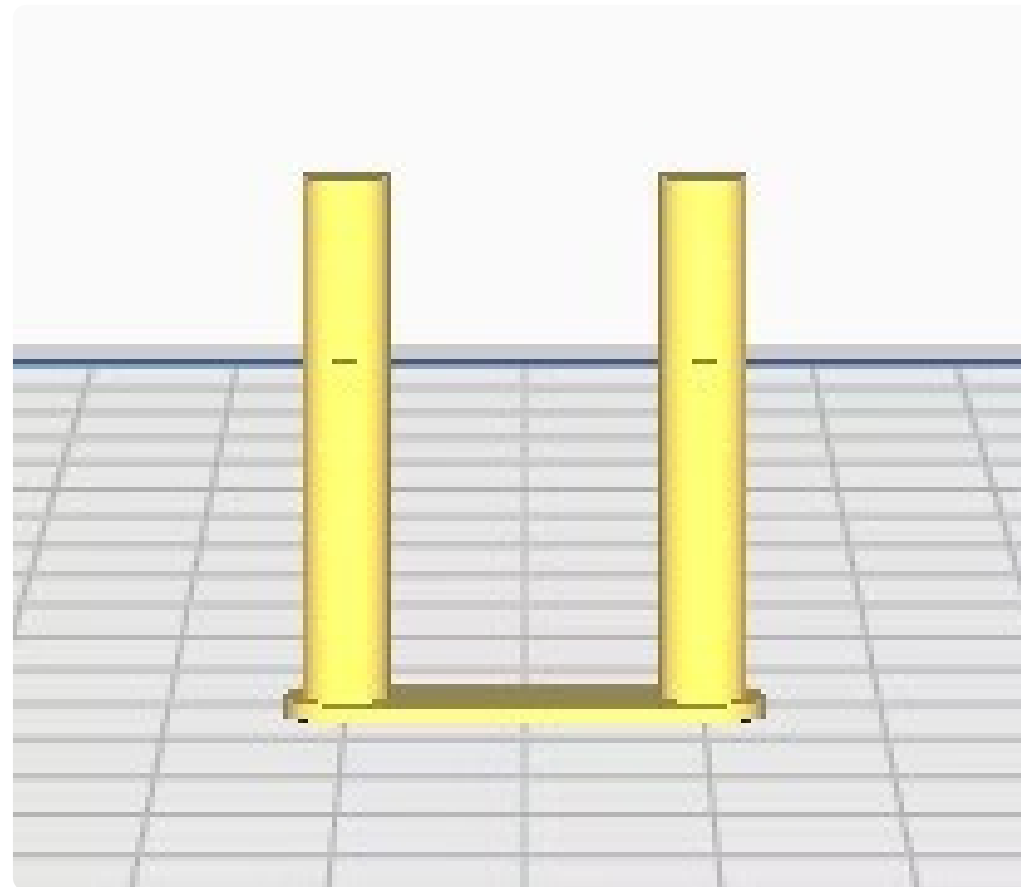
Experimental Setup — Controlled Defect Induction

The Ender 3 was deliberately misconfigured with four parameter changes to **guarantee producible Stringing artifacts** visible to the camera during test prints.

Parameter	Default	Modified	Reason
Nozzle Temperature	200°C	225°C	Keeps filament molten causes ooze during travel moves
Retraction	45 mm	0 mm (Off)	No pullback = maximum stringing between features
Travel Speed	50 mm/s	40 mm/s	Slower travel gives molten filament more time to drip
Cooling Fan	100%	30%	Less cooling keeps filament fluid longer, worsening stringing
Slicing & G-code Generation	Standard Cura settings	Custom Cura profile with modified parameters	Used Cura to slice the 3D model and embed all modified printing parameters (nozzle temp, retraction, travel speed, fan speed) into G-code. Generated .gcode file exported to SD card and loaded into Ender-3 printer for reproducible test prints



Ender 3 Printer



Object to be printed

Hardware & Network Setup

Using a Mobile Phone as a WiFi IP Camera

The phone streams MJPEG/HTTP video over a local WiFi LAN. Python OpenCV reads the stream via `cv2.VideoCapture(stream_url)`, treating it exactly like a local webcam **zero additional hardware cost**.



→ IP Camera App

Installed on phone; broadcasts stream at
`http://192.168.x.x:8080/video`

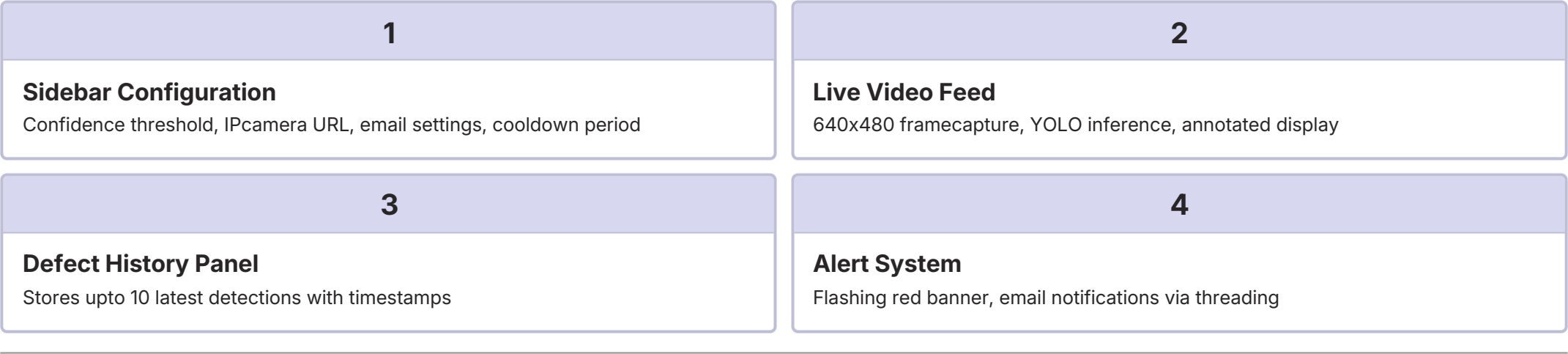
→ Positioning

Phone mounted to capture the full Ender 3 build plate and stringing artifacts at an angle

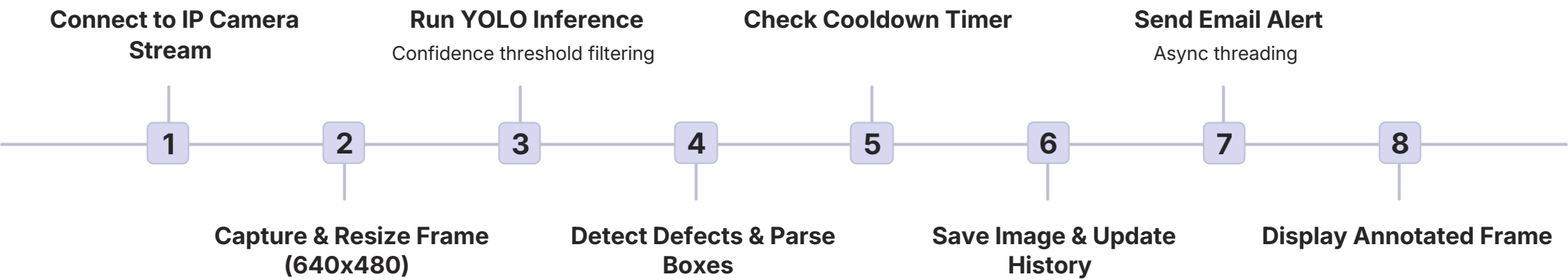
→ Advantage

Leverages existing consumer devices; easy repositioning without dedicated hardware

Streamlit Web Application



Workflow & Data Flow



Model & Setup

```
import streamlit as st
import cv2, os, time, threading, smtplib
from email.message import EmailMessage

st.set_page_config(layout="wide")

@st.cache_resource
def load_model():
    return YOLO("model.pt")

model = load_model()

if "monitoring" not in st.session_state:
    st.session_state.monitoring = False
if "defect_history" not in st.session_state:
    st.session_state.defect_history = []
if "last_alert_time" not in st.session_state:
    st.session_state.last_alert_time = 0
```

Core Logic

```
def send_email_alert(subject, body, sender, password, receiver, image_path):
    def _send():
        msg = EmailMessage()
        msg.update({"Subject": subject, "From": sender, "To": receiver})
        msg.set_content(body)
        with open(image_path, "rb") as f:
            msg.add_attachment(f.read(), maintype="image", subtype="jpeg",
                               filename=os.path.basename(image_path))
        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
            smtp.login(sender, password)
            smtp.send_message(msg)
        threading.Thread(target=_send, daemon=True).start()

    if len(results[0].boxes) > 0 and time.time() - st.session_state.last_alert_time >= alert_cooldown:
        highest_confidence = max(results[0].boxes.conf.tolist())
        image_path = f"defects/defect_{int(time.time())}.jpg"
        cv2.imwrite(image_path, cv2.cvtColor(annotated_frame,
                                              cv2.COLOR_RGB2BGR))
        send_email_alert("Defect Detected", f"A defect was detected.",
                        sender_email, sender_password, receiver_email, image_path)
        st.session_state.last_alert_time = time.time()
```

Live Deployment Results

Settings

IP Camera URL (e.g., <http://192.168.1.xxx:8080/video>)

<http://10.96.242.190:8080/video>

Email Alert Setup

For Gmail, use an App Password instead of your regular password.

Sender Email

ssenapati721@gmail.com

Sender App Password

.....

Receiver Email

122me0914@nitrkl.ac.in

Alert Cooldown (seconds)

60

Start Monitoring

Stop Monitoring

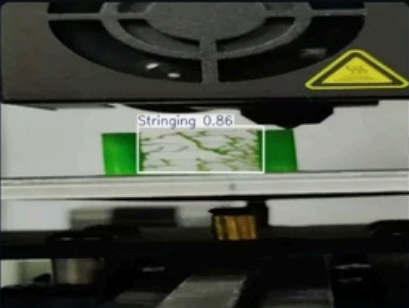
3D Printing Defect Monitor

Live Video Feed



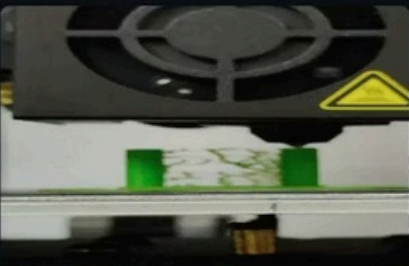
Stringing 0.42

Defect History



Stringing 0.86

2026-05-09 16:37:15 - STRINGING



Stringing 0.86

Live Deployment Results

The system proved reliable in a real printing environment. On an Ender 3 with induced stringing, it detected the target defect without overreacting to normal variation. It stayed stable through multiple runs, changing surface conditions, and real-world noise.

It ignored brief artifacts, lighting shifts, and minor print fluctuations. It only raised concern when stringing became persistent enough to signal a true issue. That made the alerts far more trustworthy.

Operational success

The deployment completed successfully and produced clear, timely defect signals. Alerts clustered around the moments when stringing became visually pronounced, matching the observed print behavior.

Practical monitoring

This makes the system practical for real monitoring, not just offline inspection. It is predictable, low-overhead, and suitable for everyday quality-control workflows.

Robust enough for scale

Overall, the Ender 3 validation shows the approach is robust enough for workshops, labs, and small-scale manufacturing.

Conclusion & Future Roadmap

✓ What We Achieved

- Upgraded model from YOLOv26m → **YOLOv26l** for improved detection depth
- Validated on a real Ender 3 with **intentionally induced Stringing**
- Built a **complete closed-loop pipeline** using only consumer grade hardware
- Delivered **automated email alerts** with snapshots no human monitoring required

Future Roadmap

G-code Integration

Send M600 pause or M112 emergency stop via USB on defect detection

Multi-Defect Validation

Live-test all 5 classes: Blobs, Spaghetti, Under- Extrusion, Warping, Stringing

Edge Deployment

Port to Raspberry Pi or Jetson Nano for standalone, always-on monitoring

Web Dashboard

Live feed, detection history, and print health status in a browser interface

Thank You!